

Calcolatori Elettronici

Gennaio 2025

Indice

1 Domande ed Esercizi	8
1.1 Domande	8
1.1.1 A cosa serve il registro MAR nel DLX? Quali istruzioni modificano tale registro?	8
1.1.2 Esistono istruzioni che modificano il contenuto del registro PC nel DLX? In caso affermativo, quali? In caso negativo, perché?	8
1.1.3 A cosa servono i registri A e B nel DLX sequenziale? Motivare chiaramente la risposta data al punto precedente.	8
1.1.4 Esiste una differenza tra indirizzo logico e indirizzo fisico nel DLX? Motivare chiaramente la risposta fornita al punto precedente.	8
1.1.5 Nella rete che realizza il DLX, è possibile utilizzare un PC a 30 bit? Sì o No?	9
1.1.6 Come è possibile eliminare la ALU nello stadio di fetch del DLX pipelined? Fornire dettagli in merito alla risposta fornita al punto precedente.	9
1.1.7 A cosa serve il segnale di READY in un processore come il DLX? Spiegare come agisce tale segnale.	9
1.1.8 È possibile eseguire un'istruzione jump and link all'interno di un interrupt handler? Sì o no? Motivare chiaramente e sinteticamente la risposta al punto precedente.	9
1.1.9 Nel DLX, sono presenti vincoli nell'accesso in memoria a word? Sì o No? Motivare chiaramente e sinteticamente la risposta al punto precedente.	10
1.1.10 Spiegare quale strategia è necessario adottare per evitare inconsistenze tra i dati nella scrittura di un interrupt handler motivando chiaramente la risposta.	10
1.1.11 Nel DLX, sarebbe possibile non generare BE0, BE1, BE2, BE3 e utilizzare in modo equivalente unicamente i segnali BA0 e BA1? Sì o no? In caso di risposta affermativa, come? In caso contrario, perché?	10
1.1.12 È possibile eseguire una JAL all'interno di codice chiamato da un'altra JAL? Sì o no?	10
1.1.13 Indicare chiaramente e sinteticamente quali sono le motivazioni per utilizzare il delayed load. Esistono controindicazioni nell'utilizzo di tale strategia?	11

1.1.14	Quali sono i vantaggi nell'utilizzo del protocollo di handshake rispetto a non utilizzarlo. Rispondere in modo sintetico motivando chiaramente la risposta.	11
1.1.15	In una CPU pipelined: come sono eliminate le istruzioni erroneamente inserite nella pipeline di elaborazione nel caso la predizione di un salto risulti errata?	11
1.1.16	Con riferimento al forwarding: spiegare qual è la specifica funzione del multiplexer collocato prima della barriera ID/EX.	11
1.1.17	Indicare chiaramente quali sono le controindicazioni nell'utilizzo della tecnica del delayed branch.	12
1.1.18	Quali problemi risolve la tecnica del cycle-split? In cosa consiste e quali problemi comporta? E' necessario utilizzarla necessariamente o esistono alternative (e quali)?	12
1.1.19	Quali sono i vantaggi derivanti dalla particolare codifica delle istruzioni nel DLX?	12
1.1.20	In cosa consiste il delayed branch? Perché potrebbe non essere la scelta ottimale nella progettazione di una CPU?	13
1.1.21	È possibile annidare interruzioni nel DLX? In caso affermativo, come? In caso negativo, perché?	13
1.1.22	Quali strategie è opportuno adottare nella gestione dei registri nella scrittura di un interrupt handler? Cosa può accadere se non si adottano tali strategie?	13
1.1.23	Mostrare e descrivere le forme d'onda di una porta in OUTPUT che comunica con l'esterno mediante il protocollo di handshake.	13
1.1.24	Per quale finalità è presente il registro IAR nel DLX sequenziale?	14
1.1.25	Per quale ragione è bene non sovrascrivere il registro R31?	14
1.1.26	Per quali ragioni si preferisce utilizzare un Branch Target Buffer piuttosto che il delayed branch per gestire le alee di controllo?	14
1.1.27	Qual è la dimensione dell'immediato nelle istruzioni di load e store nel DLX? In quale modo è esteso tale immediato? Con quanti bit è codificato il valore esteso? .	15
1.1.28	Con riferimento all'ISA del DLX: A cosa serve l'istruzione RFE? Che cosa implica l'esecuzione di tale istruzione?	15

1.1.29	Strategie (hardware e software) per gestire le alee di controllo	15
1.1.30	Con riferimento all'ISA del DLX: Con quanti bit è codificato l'immediato nelle istruzioni di load e store? Com'è esteso il valore di tale immediato? Com'è calcolato l'indirizzo di accesso alla memoria e con quanti bit è codificato tale indirizzo?	16
1.1.31	A cosa serve una forwarding unit? Che tipo di rete è? Quali informazioni elabora la forwarding unit?	17
1.1.32	Quale funzione svolge il Branch Target Buffer? In quale stadio agisce?	17
1.1.33	Nel DLX, unicamente dagli indirizzi emessi dal processore, è possibile determinare i segnali BE _i , i=0,1,2,3 ? Si o No? Se Si, come? Se No, perché?	17
1.1.34	Quale problema mira a risolvere lo Split Cycle? Come agisce? Esistono alternative?	17
1.1.35	In cosa consiste un branch target buffer? In quale stadio agisce? Come può essere reso più efficace in caso di loop?	18
1.1.36	Che problema mira a risolvere la tecnica del Delayed Load? Come agisce?	18
1.1.37	Quali sono e come possono essere eliminate le alee strutturali nel DLX pipelined?	18
1.1.38	In un sistema con bus dati a 32 bit come il DLX, è possibile ottenere i segnali BE ₀ , BE ₁ , BE ₂ e BE ₃ dai segnali BA ₀ e BA ₁ ?	19
1.1.39	Quali problemi implicano le alee di dato generate da letture in memoria e come sono gestite nel DLX? . . .	19
1.1.40	Le istruzioni di scrittura in memoria di dati signed e unsigned sono differenziate nel DLX? Si o no? Motivare, sinteticamente e chiaramente, la risposta.	19
1.1.41	Indicare quali potrebbero essere (tutti) i segnali d'ingresso e di uscita della rete logica Register File utilizzato dal DLX. Per ogni segnale si aggiunga una breve descrizione.	20
1.1.42	Che cosa s'intende per architettura Harvard? Quali problemi mira a risolvere e come può essere realizzata nel caso del DLX?	20
1.1.43	Spiegare cosa sono le alee di controllo e quali strategie esistono per poterle gestirle	20

1.1.44	Spiegare la funzione del Register File nel DLX indicando chiaramente quali sono tutti i segnali di input e output di tale modulo. Descrivere in dettaglio la struttura interna di un register file	21
1.1.45	Quali sono le fasi comuni a tutte le istruzioni definite mediante il diagramma degli stati del controller? Indicare possibili varianti e le conseguenti modifiche hardware alla struttura del DLX sequenziale	22
1.1.46	Cosa si intende per mapping di memorie a indirizzi non allineati? Spiegare in dettaglio, con un esempio, come può essere realizzato.	22
1.1.47	Quali soluzioni software esistono per gestire le alee di controllo? E' necessario impostare l'unità di controllo in modo opportuno?	23
1.1.48	Esistono nel DLX diverse tipologie di istruzioni per scritture in memoria di halfword di tipo signed e unsigned? Si o No? Motivare chiaramente la risposta . . .	23
1.1.49	Quali informazioni sono necessarie all'unità di controllo del DLX per generare i segnali BEi, con i=[0,1,2,3]? Motivare chiaramente la risposta.	24
1.1.50	Descrivere l'organizzazione della memoria e i segnali emessi dal processore nel caso di un sistema con bus dati a 64 bit.	24
1.1.51	Spiegare per quale motivo è più efficace utilizzare nel Branch Target Buffer un meccanismo di predizione basato su due bit rispetto a una soluzione con un singolo bit.	24
1.1.52	Descrivere chiaramente per quale ragione è possibile utilizzare un contatore per lo stadio di fetch del DLX pipelined e quali caratteristiche deve possedere tale contatore.	24
1.1.53	Spiegare chiaramente cosa sono le eccezioni in un sistema a microprocessore	25
1.1.54	Spiegare cosa è la metastabilità e quali metodologie possono essere adottate per evitarne l'origine.	25
1.1.55	Spiegare le differenze tra la gestione dell'Input/Output mediante polling e interrupt evidenziando vantaggi e svantaggi di ciascuna metodologia.	25
1.1.56	Indicare come è possibile gestire la condizione di stallo di uno stadio di un processore pipelined.	26

1.1.57	Con riferimento allo schema del DLX sequenziale mostrato in aula, è possibile effettuare modifiche per ridurre il tempo necessario per eseguire il fetch delle istruzioni?	26
1.1.58	Con riferimento al DLX pipelined: come sono gestiti gli accessi alla memoria di più basso livello nel caso entrambe le memorie IM e DM non contengono le informazioni richieste?	26
1.1.59	Con riferimento a istruzioni PC-relative come i branch; eseguendo del codice, potrebbero verificarsi casi nei quali il DLX a non è in grado di portare a termine l'esecuzione di tali istruzioni?	26
1.1.60	Nel DLX, come avviene l'estensione di un byte signed letto dalla memoria? Spiegare in dettaglio	27
1.1.61	In un sistema reale basato su DLX è sempre necessaria la rete logica che consente di determinare se il sistema è in fase di avvio?	27
1.2	Esercizi di codice	28
1.2.1	Sostituire alla label HERE, il valore esadecimale corrispondente	28
1.2.2	Sostituire alla label DEST, il valore esadecimale corrispondente:	28
1.2.3	Il codice seguente è corretto? Motivare chiaramente la risposta.	29
1.2.4	In un DLX senza forwarding unit/cycle-split, indicare quanti stalli risultano con la seguente sequenza di istruzioni mostrando graficamente l'evoluzione temporale della pipeline.	29
1.2.5	Determinare il numero di stalli che si verificano con la seguente sequenza di istruzioni con processore DLX pipelined studiato a lezione in due casi: a) Senza forwarding unit b) Con forwarding unit	29
1.2.6	Indicare quali strategie, hardware e software, è possibile adottare per risolvere eventuali alee generate dalla sequenza di istruzioni seguente:	30

1.2.7	Si consideri la seguente sequenza di istruzioni assembler DLX. Indicare, mostrando l'evoluzione delle istruzioni nella pipeline, quanti stalli risultano nei seguenti tre casi: senza forwarding unit e senza cycle-split, con forwarding unit e senza cycle-split, senza forwarding unit e con cycle-split	31
1.2.8	Con riferimento al DLX pipelined non dotato di unità di cycle split; indicare quanti stalli risultano nell'esecuzione delle seguenti istruzioni: senza forwarding unit, con forwarding unit	32
1.2.9	Quale valore assume il registro R2 al termine dell'esecuzione? Quanto vale lo skip?	32

1 Domande ed Esercizi

1.1 Domande

1.1.1 A cosa serve il registro MAR nel DLX? Quali istruzioni modificano tale registro?

Il registro *MAR* all'interno del DLX viene utilizzato per salvare l'indirizzo al quale è stato effettuato l'ultimo accesso in memoria.

Ogni istruzione ne modifica il contenuto durante la fase di fetch ma solo due macro-gruppi ne cambiano nuovamente il contenuto in fasi successive: *STORE* e *LOAD*. Quest'ultime, in tutte le loro varianti, necessitano di ulteriori accessi alla memoria per essere completate

1.1.2 Esistono istruzioni che modificano il contenuto del registro PC nel DLX? In caso affermativo, quali? In caso negativo, perché?

Sì, esistono operazioni che modificano il contenuto del *Program Counter*. Le operazioni di salto, infatti, modificano il valore del registro PC al fine di modificare il flusso del programma, che queste siano condizionate (*BRANCH*) o non condizionate (*JUMP* & *JUMP AND LINK*)

1.1.3 A cosa servono i registri A e B nel DLX sequenziale? Motivare chiaramente la risposta data al punto precedente.

I registri A e B nel DLX sequenziale vengono utilizzati per trasportare i valori dei registri *General Purpose* utilizzati dall'istruzione prossima alla fase di *EXECUTE* nella fase stessa. Questi registri di "trasporto" vengono utilizzati in maniera speculativa poiché prelevati a prescindere durante la fase di *DECODE*, che i bit codificati dell'istruzione indicassero loro o meno

1.1.4 Esiste una differenza tra indirizzo logico e indirizzo fisico nel DLX? Motivare chiaramente la risposta fornita al punto precedente.

Esiste una differenza tra indirizzo logico e fisico nel DLX. L'indirizzo fisico rappresenta l'indirizzo hardware in cui risiede una memoria o un qualunque dispositivo connesso al bus dati. L'indirizzo logico, invece, rappresenta la posizione "virtuale" all'interno di uno spazio di indirizzamento che tiene in considerazione il parallelismo del DLX.

1.1.5 Nella rete che realizza il DLX, è possibile utilizzare un PC a 30 bit? Sì o No?

Sì, è possibile utilizzare un *Program Counter* (PC) a 30 bit. Le operazioni DLX sono rappresentate con delle WORD, per cui allineate sempre a 4 byte. Questo implica la costante presenza di 0 nei due bit meno significativi del PC, superflui.

1.1.6 Come è possibile eliminare la ALU nello stadio di fetch del DLX pipelined? Fornire dettagli in merito alla risposta fornita al punto precedente.

È possibile rimuovere la ALU dallo stadio di *FETCH* sostituendo il *Program Counter* con un contatore a 30 bit anziché un registro. Il contatore verrebbe abilitato dal segnale di stallo negato ($EN = \text{STALL}'$) inviatogli dalla *Control Unit*, ne verrebbe modificato il contenuto attraverso un segnale di jump ($LD = \text{JUMP}$, $LD[29..0]$ varia in base a salti condizionati e non condizionati) e ne sarebbe attivato il conteggio solamente in avanti.

1.1.7 A cosa serve il segnale di READY in un processore come il DLX? Spiegare come agisce tale segnale.

READY è un segnale generato dal *Memory Controller* utilizzato dai processori per indicare la possibilità, o meno, di concludere un'operazione che coinvolge componenti esterni al processore stesso, per cui più lenti. Il segnale di READY, infatti, condiziona la durata di un ciclo di bus e ne assicura il corretto risultato. In un caso di lettura, per esempio, il dato non viene considerato presente sul BD fino all'arrivo del segnale di READY

1.1.8 È possibile eseguire un'istruzione jump and link all'interno di un interrupt handler? Sì o no? Motivare chiaramente e sinteticamente la risposta al punto precedente.

Sì, è possibile effettuare un'operazione di JAL all'interno di un interrupt handler a patto che si tenga in considerazione il possibile utilizzo del registro R31 da parte del programma "main". Il *Program Counter* precedente all'interrupt non verrebbe perso perché salvato all'interno dell'*IAR*

1.1.9 Nel DLX, sono presenti vincoli nell'accesso in memoria a word? Sì o No? Motivare chiaramente e sinteticamente la risposta al punto precedente.

Sì, nel DLX sono presenti vincoli per l'accesso in memoria a *WORD*. Per leggere/scrivere una *WORD*, infatti, è necessario che l'indirizzo di provenienza/destinazione sia allineato a 4 Byte (la lunghezza di una *WORD* stessa). Se così non fosse servirebbero più cicli di bus per accedere al dato.

1.1.10 Spiegare quale strategia è necessario adottare per evitare inconsistenze tra i dati nella scrittura di un interrupt handler motivando chiaramente la risposta.

Per evitare inconsistenza tra i dati dovuta ad un *Interrupt Handler* è necessario che questo salvi il contenuto dei registri da utilizzare, in memoria, per poi ripristinarne il valore prima dell'operazione di RFE. Se così non facesse, al ritorno dall'handler il programma "main" potrebbe aver perso i dati con i quali stava lavorando.

1.1.11 Nel DLX, sarebbe possibile non generare BE0, BE1, BE2, BE3 e utilizzare in modo equivalente unicamente i segnali BA0 e BA1? Sì o no? In caso di risposta affermativa, come? In caso contrario, perché?

No, non è possibile sostituire i segnali BE1, BE2, BE3 e BE4 con i segnali BA0 e BA1 poiché quest'ultimi non sarebbero in grado di descrivere tutti i possibili accessi alla memoria. I BA0 e BA1, infatti, sarebbero in grado di fornire 4 possibili configurazioni diverse per ogni *WORD* che permetterebbero, per esempio, solo l'accesso ai singoli byte della *WORD* stessa, escludendo le altre possibili combinazioni. Si andrebbe a perdere, di fatto, il parallelismo.

1.1.12 È possibile eseguire una JAL all'interno di codice chiamato da un'altra JAL? Sì o no?

Eseguire una JAL all'interno del codice chiamato da un'altra JAL è teoricamente possibile ma è importante tenere in considerazione la sovrascrittura (e quindi eventualmente la necessità di attuare opportune tecniche di salvataggio) del registro R31, il quale contiene l'indirizzo di partenza del primo JAL.

1.1.13 Indicare chiaramente e sinteticamente quali sono le motivazioni per utilizzare il delayed load. Esistono controindicazioni nell'utilizzo di tale strategia?

La tecnica del *Delayed Load* consiste, attraverso il compilatore, nello spostamento di operazioni indipendenti e precedenti alla *LOAD* al suo seguito. Qualora non ci fossero operazioni adatte a tale modifica, verrebbero inseriti dei NOP.

Questa strategia risulta particolarmente tortuosa perché dipendente dal numero di stati presenti nella pipeline e, soprattutto, non utile nel caso di inserimento di NOP, poiché pari a stalli.

1.1.14 Quali sono i vantaggi nell'utilizzo del protocollo di handshake rispetto a non utilizzarlo. Rispondere in modo sintetico motivando chiaramente la risposta.

I vantaggi dell'utilizzo del protocollo di handshake risiedono nella sicurezza e affidabilità del trasferimento dei dati tra la CPU e il dispositivo. Tale protocollo, infatti, permette al dispositivo di comunicare alla CPU il verificarsi di eventi da gestire, senza la necessità che questa lo verifichi periodicamente, e garantisce che il dispositivo sia pronto prima di inviare o ricevere dati.

1.1.15 In una CPU pipelined: come sono eliminate le istruzioni erroneamente inserite nella pipeline di elaborazione nel caso la predizione di un salto risulti errata?

Nel caso in cui la predizione di un salto risulti errata, le operazioni già presenti all'interno della pipeline vengono sostituite da dei NOP (no-operation) dalla *Control Unit*.

1.1.16 Con riferimento al forwarding: spiegare qual è la specifica funzione del multiplexer collocato prima della barriera ID/EX.

In caso di forwarding, il multiplexer collocato prima della barriera ID/EX permette di fornire il valore corretto del registro che si sta per utilizzare, anche se non ancora aggiornato nel *Register File*.

Il multiplexer ha due ingressi principali:

- Il valore del registro prelevato dal *Register File*.
- Il valore risultato dalla fase di WB.

Il selettore del multiplexer è controllato dalla *Forwarding Unit* (FU), la quale governa tutte le operazioni di propagazione.

Questa operazione risulta necessaria poiché il *Register File* lavora sul fronte di salita del clock, che implica l'aggiornamento dei valori dei registri solo nella fase di clock successiva a quella di WB.

1.1.17 Indicare chiaramente quali sono le controindicazioni nell'utilizzo della tecnica del delayed branch.

Le controindicazioni della tecnica di *Delayed Branch* sono legate alla sua stretta connessione con l'hardware (ovvero il numero di stadi della pipeline prima della valutazione del **BRANCH**) e alla difficile individuazione delle istruzioni adatte allo spostamento al seguito del **BRANCH** stesso.

1.1.18 Quali problemi risolve la tecnica del cycle-split? In cosa consiste e quali problemi comporta? E' necessario utilizzarla necessariamente o esistono alternative (e quali)?

Il *cycle-split* è una tecnica che consiste nella divisione di ogni ciclo di Clock in due distinte fasi e risolve i problemi legati alla lettura e scrittura del *Register File*. All'atto pratico lega la scrittura sul *Register File* alla prima parte di clock (ad esempio il fronte di salita) mentre la lettura alla seconda parte (ad esempio il fronte di discesa), il che comporta la necessità di componenti più reattivi oppure, nei casi nei quali non è possibile soddisfare questa condizione, un ciclo di clock di durata superiore.

Se si vogliono ridurre i casi di stallo la tecnica del cycle-split può rappresentare una soluzione, ma non l'unica. Esistono, ad esempio, il *Forwarding* oppure l'*Esecuzione speculativa*.

1.1.19 Quali sono i vantaggi derivanti dalla particolare codifica delle istruzioni nel DLX?

I vantaggi derivanti dalla particolare codifica delle istruzioni del DLX risiedono nella scelta di mantenere i dati che rappresentano le informazioni comuni sempre sugli stessi bit. L'*OPCODE* è rappresentato sempre dagli stessi bit (indipendentemente dal fatto che si tratti di operazioni R,I oppure J), così come il registro di destinazione nel caso di operazione R ed I.

Questo permette, ad esempio, la speculazione sull'utilizzo dei registri prima ancora che l'operazione arrivi alla fase di *EXECUTE* oppure l'individuazione semplificata dell'operazione che si vuole andare a svolgere.

1.1.20 In cosa consiste il delayed branch? Perché potrebbe non essere la scelta ottimale nella progettazione di una CPU?

Con la tecnica del *Delayed Branch* si affida al compilatore il compito di spostare quelle istruzioni indipendenti e precedenti ad un **BRANCH** al suo seguito. In questo modo, si sfruttano i *Branch Delay Slots* per eseguire istruzioni di cui si ha a priori la certezza di esecuzione.

Questa tecnica potrebbe, però, non essere ottimale poiché dipendente dalla lunghezza delle pipeline fino alla valutazione dei **BRANCH**, dalla ricerca di istruzioni valide per lo spostamento ed all'inserimento di **NOP**, qualora non fosse stato possibile effettuare spostamenti.

1.1.21 È possibile annidare interruzioni nel DLX? In caso affermativo, come? In caso negativo, perché?

Non è possibile annidare interruzioni nel DLX. Questo limite è dovuto al registro **IAR** (*Interrupt Address Register*), il quale memorizza l'indirizzo del *Program Counter* (**PC**) dell'istruzione interrotta. Questo infatti verrebbe sovrascritto al verificarsi di un'ulteriore interruzione e, di conseguenza, il valore necessario per ritornare correttamente all'istruzione interrotta originariamente verrebbe perso, impedendo il corretto ripristino all'origine.

1.1.22 Quali strategie è opportuno adottare nella gestione dei registri nella scrittura di un interrupt handler? Cosa può accadere se non si adottano tali strategie?

Quando si scrive un interrupt handler è opportuno considerare che i registri potrebbero contenere dei valori utili per l'esecuzione del programma principale. Al fine di non compromettere la normale esecuzione del codice è quindi importante, all'inizio di ogni handler, salvare in memoria il valore dei registri che verranno utilizzati, per poi ripristinarne il valore originale prima dell'**RFE**.

1.1.23 Mostrare e descrivere le forme d'onda di una porta in OUTPUT che comunica con l'esterno mediante il protocollo di handshake.

Il segnale di **WRITE** viene fornito dal DLX nei momenti in cui desidera scrivere sul dispositivo di **OUTPUT**.

OBF è un segnale di controllo (verso il dispositivo esterno) che la porta di **OUTPUT** utilizza per indicare la presenza di dati validi. Questo ha valore positivo quando il DLX finisce di scrivere.

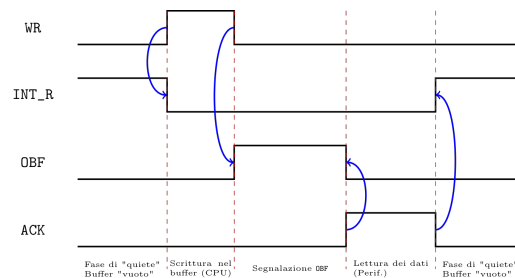


Figura 1: Immagine da lorenzoarlo.it

A seguito del segnale OBF il dispositivo esterno può rispondere con un segnale ACK che sul fronte di salita disattiva OBF e sul fronte di discesa indica la consumazione del buffer.

Il fronte di discesa di ACK viene poi seguito dal segnale di INT_R (interrupt), il quale verrà disattivato da una nuova scrittura da parte del DLX

1.1.24 Per quale finalità è presente il registro IAR nel DLX sequenziale?

Il registro IAR (*Interrupt Address Register*) viene utilizzato per salvare il valore di PC (*Program Counter*) quando si verifica un'interruzione ed è necessario effettuare un salto all'handler. Il valore di questo registro viene trasferito all'interno del PC nel momento in cui viene effettuato un RFE

1.1.25 Per quale ragione è bene non sovrascrivere il registro R31?

È importante non sovrascrivere il registro R31 poiché quest'ultimo viene utilizzato per salvare il valore del PC (*Program Counter*) nel caso di JAL (*Jump And Link*), in maniera tale da poter tornare indietro conclusa la "subroutine".

1.1.26 Per quali ragioni si preferisce utilizzare un Branch Target Buffer piuttosto che il delayed branch per gestire le alee di controllo?

Il *Branch Target Buffer* è preferito al *Delayed Branch* per gestire le alee di controllo perché utilizza una predizione dinamica dei salti basata sulla storia dei **BRANCH**, permettendo di ridurre i cicli di stallo e mantenere alte prestazioni anche con pipeline profonde. Inoltre, non richiede interventi del compilatore, adattandosi a qualunque pipeline.

Il *Delayed Branch*, invece, è una tecnica statica, perché dipendente dal com-

pilatore per riempire i *Branch Delay Slots*, ed inefficace con pipeline lunghe o in caso di carenza di istruzioni indipendenti.

1.1.27 Qual è la dimensione dell'immediato nelle istruzioni di load e store nel DLX? In quale modo è esteso tale immediato? Con quanti bit è codificato il valore esteso?

Gli immediati utilizzati nelle istruzioni di LOAD e STORE nel DLX hanno una dimensione di 16 bit. Questi vengono estesi a 32 bit utilizzando l'estensione del segno, concatenando il bit più significativo Imm[15] per 16 volte ai 16 bit originali:

$$(\text{Imm}[15])^{16} \# \# \text{Imm}[15..0]$$

. Il valore così esteso a 32 bit viene successivamente sommato al valore contenuto in un registro specificato nell'istruzione, determinando l'indirizzo finale per l'operazione di memoria.

1.1.28 Con riferimento all'ISA del DLX: A cosa serve l'istruzione RFE? Che cosa implica l'esecuzione di tale istruzione?

L'istruzione RFE indica la fine dell'interrupt handler per il ritorno al normale flusso del programma. In particolare, implica l'inserimento all'interno del PC del valore salvato all'interno del registro IAR, nonché la riattivazione degli interrupt in ingresso.

1.1.29 Strategie (hardware e software) per gestire le alee di controllo

Per affrontare le alee di controllo, esistono diverse strategie che sfruttano sia soluzioni hardware che software. Di seguito vengono elencate con una breve descrizione del loro principio di funzionamento:

- **Politica *Predict not taken*:** Questa strategia hardware assume che il BRANCH non venga preso (*not taken*). Di conseguenza, inserisce nella pipeline le istruzioni successive al BRANCH.
 - Se il BRANCH risulta *not taken*, le istruzioni continuano ad essere eseguite normalmente.
 - Se il BRANCH risulta *taken*, le istruzioni inserite vengono invalidate e sostituite con *NOP*, introducendo cicli di stallo.

- **Valutazione BRANCH in EXECUTE:** Questa strategia hardware anticipa la valutazione della condizione del **BRANCH** e la sostituzione del PC alla fase di *EXECUTE* della pipeline
 - Quando la condizione del **BRANCH** viene determinata, viene aggiornato il Program Counter (PC) usando un multiplexer, che inserisce l'indirizzo corretto.
 - Le istruzioni erroneamente fetchate e decodificate prima della valutazione del **BRANCH** vengono sostituite con *NOP*.
- **Delayed Branch:** Questa strategia software richiede l'intervento del compilatore per riordinare le istruzioni.
 - Le istruzioni indipendenti e immediatamente precedenti al **BRANCH** vengono spostate dopo di esso.
 - Mentre la condizione del **BRANCH** viene valutata, le istruzioni "di riempimento" possono essere eseguite nella pipeline, riducendo i cicli di stallo.
- **Branch Target Buffer (BTB):** Questa è una strategia hardware che utilizza una memoria associativa per memorizzare i risultati recenti dei **BRANCH**.
 - Ogni entry del BTB registra l'indirizzo del **BRANCH** e l'indirizzo target, oltre alla storia del risultato (*taken* o *not taken*).
 - Quando un **BRANCH** viene incontrato, il BTB prova a predirne l'esito e inserisce nella pipeline le istruzioni.
 - In caso di predizione errata, le istruzioni fetchate vengono scartate e sostituite con quelle corrette.

1.1.30 Con riferimento all'ISA del DLX: Con quanti bit è codificato l'immediato nelle istruzioni di load e store? Com'è esteso il valore di tale immediato? Com'è calcolato l'indirizzo di accesso alla memoria e con quanti bit è codificato tale indirizzo?

Gli immediati utilizzati nelle istruzioni di **LOAD** e **STORE** nel DLX hanno una dimensione di 16 bit. Questi vengono estesi a 32 bit utilizzando l'estensione del segno, cioè concatenando il bit più significativo, per 16 volte, ai 16 bit originali:

$$(\text{Imm}[15])^{16} \# \# \text{Imm}[15..0]$$

. Il valore così esteso (a 32 bit) viene successivamente sommato al valore contenuto in un registro specificato nell'istruzione, determinando l'indirizzo finale per l'operazione di memoria.

1.1.31 A cosa serve una forwarding unit? Che tipo di rete è? Quali informazioni elabora la forwarding unit?

La *FORWARDING UNIT* è una rete logica presente nei DLX pipelined utilizzata per risolvere le alee di dato legate alle operazioni RAW (*Read After Write*). In particolare, si preoccupa di sostituire i valori dei registri attualmente in uso nelle fasi di DECODE ed EXECUTE della pipeline con la loro versione aggiornata ma non ancora presente nel *Register File* o comunque non presente nel momento dell'estrazione dei dati poiché ancora nelle fasi di MEM o WB.

Da quest'ultimi riceve in input l'OPCODE dell'istruzione ed il registro di destinazione. Su questi vengono calcolati gli indirizzi di controllo dei mux degli stati sopraccitati.

1.1.32 Quale funzione svolge il Branch Target Buffer? In quale stadio agisce?

Il *Branch Target Buffer* è una "tabella" con lo scopo di prevedere il risultato di un BRANCH dato dai precedenti risultati dello stesso. Essa agisce nella fase di FETCH e modifica l'indirizzo delle operazioni in ingresso (quelle nella posizione del jump se si prevede *taken*, le successive altrimenti)

1.1.33 Nel DLX, unicamente dagli indirizzi emessi dal processore, è possibile determinare i segnali BEi, i=0,1,2,3 ? Si o No? Se Si, come? Se No, perché?

No, dagli indirizzi emessi dal processore non è possibile determinare i segnali BEi. Gli indirizzi emessi, infatti, sono sempre allineati a 4 byte ma con i BEi è possibile anche accedere solamente ad un byte (non obbligatoriamente il primo della WORD). Si ricorda che gli indirizzi uscenti dal DLX sono a 30 bit.

1.1.34 Quale problema mira a risolvere lo Split Cycle? Come agisce? Esistono alternative?

Lo *Split Cycle* cerca di ovviare alla discordanza dei dati presenti nella fase di WB con quelli che sono prossimi alla lettura del *Register File* dividendo ogni ciclo di clock in due fasi: una prima nella quale si applicano le modifiche

al *Register File* ed una seconda in cui ne viene letto il contenuto. Questa tecnica presenta problematiche relative alla "prontezza" dei componenti. Se questi non dovessero essere abbastanza veloci allora sarebbe necessario aumentare la durata del clock e, di fatto, diminuire le prestazioni del sistema.

Un'alternativa valida è rappresentata dal *Forwarding*

1.1.35 In cosa consiste un branch target buffer? In quale stadio agisce? Come può essere reso più efficace in caso di loop?

Il *Branch Target Buffer* è una memoria cache associativa che memorizza informazioni sui salti condizionali (BRANCH) durante l'esecuzione del programma. Queste informazioni includono l'indirizzo del BRANCH, la destinazione del salto e la predizione calcolata. Il *Branch Target Buffer* consente al processore di predire con maggiore precisione l'indirizzo successivo da eseguire, riducendo gli stalli nella pipeline. Esso agisce nella fase di FETCH e modifica l'indirizzo delle operazioni in ingresso (quelle nella posizione del *Branch Target Address* se si prevede *taken*, le successive altrimenti).

In caso di loop è possibile rendere questa tecnica più efficiente modificando la predizione nel *Branch Target Buffer* solamente al presentarsi di due previsioni consecutive errate

1.1.36 Che problema mira a risolvere la tecnica del Delayed Load? Come agisce?

Il *Delayed Load* mira a risolvere le alee di controllo dovute alla presenza di saldi condizionati. Questa tecnica consiste, da parte del compilatore, nello spostamento delle operazioni indipendenti e precedenti al BRANCH al suo seguito, in maniera tale da riempire i *Branch Delay Slots*. Se non dovessero esserci operazioni valide a tale spostamento si inseriscono dei *NOP* a seguire del BRANCH

1.1.37 Quali sono e come possono essere eliminate le alee strutturali nel DLX pipelined?

Un DLX pipelined presenta principalmente 2 alee strutturali:

- Accesso alla memoria centrale concorrente da parte di più stati contemporaneamente
- Utilizzo della *ALU* per incrementare il *Program Counter*

L'accesso alla memoria concorrente può essere eliminato utilizzando l'*Architettura Harvard* nella progettazione, ovvero inserendo delle memorie *cache* distinte per codice e dati (eliminando il conflitto).

La necessità di una *ALU* da parte del *Program Counter* (PC) può essere risolta utilizzando un semplice *adder* per l'incremento oppure sostituendo con un contatore il registro

1.1.38 In un sistema con bus dati a 32 bit come il DLX, è possibile ottenere i segnali BE0, BE1, BE2 e BE3 dai segnali BA0 e BA1?

No, in un sistema DLX i segnali *BA0* e *BA1* non sono sufficienti per ottenere i segnali *BE* poiché è possibile effettuare molteplici operazioni partendo dallo stesso indirizzo in memoria. Se accedessimo ad un indirizzo allineato a 4 bytes, per esempio, potremmo star leggendo semplicemente un byte così come potremmo star leggendo una half-word oppure una word (ovvero 3 configurazioni differenti di BE).

1.1.39 Quali problemi implicano le alee di dato generate da letture in memoria e come sono gestite nel DLX?

Le alee di dato successive ad un'operazione di *LOAD* implicano la presenza di stalli, vista la possibile mancanza del dato a causa dei ritardi dovuti alla comunicazione con le memorie esterne. Una possibile soluzione a questo problema viene proposta dalla tecnica *Delayed Load*, la quale affida al compilatore l'operazione di spostamento, a seguito del *LOAD*, di tutte quelle operazioni che non dipendono dalla stessa. Se non dovessero esserci istruzioni disponibili a tale spostamento si prosegue inserendo dei *NOP*

1.1.40 Le istruzioni di scrittura in memoria di dati signed e unsigned sono differenziate nel DLX? Sì o no? Motivare, sinteticamente e chiaramente, la risposta.

No, le operazioni di scrittura di dati *signed* ed *unsigned* non sono differenziate. Questo è dato dal fatto che nelle fasi di scrittura in memoria vengono considerati a prescindere solamente i bit del bus dati interessati dallo spostamento, ignorando tutti gli altri.

1.1.41 Indicare quali potrebbero essere (tutti) i segnali d'ingresso e di uscita della rete logica Register File utilizzato dal DLX. Per ogni segnale si aggiunga una breve descrizione.

I segnali in ingresso al register file:

- Clock
- RS1, ovvero il primo registro da estrarre (ricavato da IR)
- RS2, ovvero il secondo registro da estrarre (ricavato da IR)
- WR, il quale abilita la scrittura sui registri
- RD, che indica il registro che si vuole andare a modificare con i dati in arrivo (dalla fase di WB)
- Data, che contiene il nuovo valore da assegnare al registro RD (dalla fase di WB)

I segnali in uscita, invece:

- A, con il valore estratto dal registro indicato da RS1
- B, con il valore estratto dal registro indicato da RS2

1.1.42 Che cosa s'intende per architettura Harvard? Quali problemi mira a risolvere e come può essere realizzata nel caso del DLX?

Per *Architettura Harvard* si intende la separazione della memoria in due sottogruppi, con la finalità di distinguere l'immagazzinamento del codice dai dati.

Questa soluzione permette di risolvere la concorrenzialità delle fasi di *FETCH* e *MEM* all'interno del DLX pipelined, implementando un sistema di *cache* separate.

1.1.43 Spiegare cosa sono le alee di controllo e quali strategie esistono per poterle gestire

Con alee di controllo si intendono tutte quelle situazioni nelle quali la pipeline è costretta a stallare a causa della presenza di un **BRANCH**.

Al fine di gestire queste eventualità esistono molteplici approcci:

- ***Predict not taken***: si permette alle operazioni immediatamente successive al **BRANCH** di entrare sempre nella pipeline. Se la condizione dovesse poi portare ad un salto allora queste verrebbero sostituite da dei **NOP**
- ***Delayed Branch***: richiede lo sforzo del compilatore nel trovare istruzione precedenti al **BRANCH** (ma che non ne modificano il risultato) per spostarle dopo di esso, in maniera tale da farle entrare nella pipeline per riempire i *Delayed Branch Slots*. Se non dovessero esserci istruzioni adatte verrebbero inserito dei **NOP**
- ***Branch Target Buffer***: un buffer che immagazina il risultato dei **BRANCH** già incontrati al fine di prevederne il risultato (*taken* oppure *not taken*) e di conseguenza inserire nella pipeline le istruzione probabilmente corrette. Questa tecnica è dinamica poiché le previsioni possono cambiare durante l'esecuzione del programma. Se una previsione si dovesse dimostrare sbagliata, le operazioni già nella pipeline vengono sostituite con dei **NOP**

1.1.44 Spiegare la funzione del Register File nel DLX indicando chiaramente quali sono tutti i segnali di input e output di tale modulo. Descrivere in dettaglio la struttura interna di un register file

Il *Register File* è utilizzato al fine di contenere e gestire l'accesso ai 32 registri disponibili all'interno di un processore DLX. Questo è composto da molteplici segnali di ingresso e uscita:

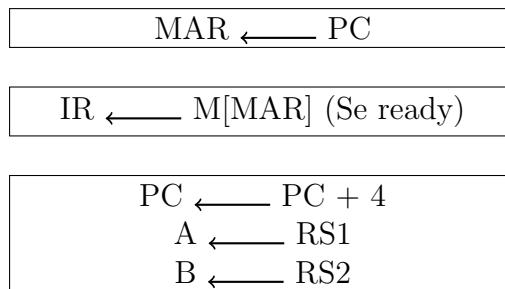
- Ingresso
 - Clock
 - RS1: primo registro da estrarre (ricavato da IR)
 - RS2: secondo registro da estrarre (ricavato da IR)
 - WR: scrittura abilitata
 - RD: registro che si vuole andare a modificare con i dati in arrivo (dalla fase di WB)
 - Data: nuovo valore da assegnare al registro RD (dalla fase di WB)
- Uscita
 - A: valore estratto dal registro indicato da RS1

- B: valore estratto dal registro indicato da RS2

In generale, la struttura interna di un register file è composta da dei *Decoder* (che individuano i registri indicati da *RS1*, *RS2* ed *RD*), registri (i veri e propri *R0*, *R1*, ... , *R31*) e multiplexer (che coordinano l'uscita dei segnali su A e B)

1.1.45 Quali sono le fasi comuni a tutte le istruzioni definite mediante il diagramma degli stati del controller? Indicare possibili varianti e le conseguenti modifiche hardware alla struttura del DLX sequenziale

Le fasi comuni a tutte le istruzioni nel diagramma degli stati sono *FETCH* e *DECODE*, o più in particolare:



Le successive fasi possono variare a seconda dell'operazione (*Data Transfer*, *ALU*, *Set*, *Jump* e *Branch*).

Al fine di soddisfare tali stati esistono due possibili implementazioni hardware del DLX:

- Sequenziale: il DLX esegue ogni istruzione stato per stato, fino al termine
- Pipelined: il DLX esegue un'istruzione per ogni stato, in maniera tale usare completamente tutte le reti. Gli stati sono separati di "blocchi" di registri che ne permettono il funzionamento autonomo

1.1.46 Cosa si intende per mapping di memorie a indirizzi non allineati? Spiegare in dettaglio, con un esempio, come può essere realizzato.

Una memoria che occupa uno spazio $\mathcal{M} = 2^k$, con k bit di indirizzo interno, si dice non allineata all'indirizzo $\mathcal{A}$ se:

$$\mathcal{A} \bmod \mathcal{M} \neq 0$$

In altre parole, un mapping di memoria si dice non allineato quando l'indirizzo di base di un'unità di memoria non è un multiplo della dimensione dello spazio che questa memoria occupa. Questo tipo di configurazione può generare complicazioni durante l'accesso alla memoria, poiché richiede un'ulteriore gestione degli indirizzi per garantire un corretto funzionamento.

Esempio: Si consideri una memoria EPROM con una capacità di 256 MB, che corrisponde a uno spazio di indirizzamento pari a 2^{28} bit ($0x10000000$ in esadecimale). Supponiamo di posizionare questa memoria all'indirizzo base $0x0000F000$.

- Per verificare se l'indirizzo è allineato, calcoliamo:

$$0x0000F000 \mod 0x10000000 \neq 0$$

Poiché il risultato non è zero, l'indirizzo $0x0000F000$ non è un multiplo dello spazio occupato dalla memoria ($0x10000000$).

- Questo indica che il mapping della memoria è **non allineato**.

1.1.47 Quali soluzioni software esistono per gestire le alee di controllo? E' necessario impostare l'unità di controllo in modo opportuno?

La soluzione software per la gestione delle alee di controllo è una:

- **Delayed Branch:** si affida esclusivamente al compilatore il compito di trovare e spostare dopo il *branch* tutte quelle operazioni che non ne modificano il risultato ma che sarebbero comunque state eseguite. Se non sono presenti operazioni di questo tipo si procede all'inserimento di NOP. Non necessita la modifica della *Control Unit*

1.1.48 Esistono nel DLX diverse tipologie di istruzioni per scritture in memoria di halfword di tipo signed e unsigned? Sì o No? Motivare chiaramente la risposta

No, nel DLX non esistono diverse tipologie di istruzioni per le scritture in memoria di halfword signed e unsigned perché non è richiesta nessuna estensione del valore che si vuole trasferire. Sulla memoria vengono trasferiti, infatti, solamente i byte indicati dai segnali BE.

1.1.49 Quali informazioni sono necessarie all'unità di controllo del DLX per generare i segnali BE_i, con $i=[0,1,2,3]$? Motivare chiaramente la risposta.

Per generare i segnali BE_i, l'unità di controllo del DLX ha bisogno di conoscere l'indirizzo di memoria e il tipo di dato (byte, half-word, word) da leggere o scrivere. L'indirizzo può determinare quale byte della word deve essere abilitato, mentre il tipo di dato definisce quanti byte abilitare. Ad esempio, se l'indirizzo termina con "10" in binario e si legge un byte, sarà necessario attivare BE₃; se si legge una half-word, si attiveranno BE₂ e BE₃.

1.1.50 Descrivere l'organizzazione della memoria e i segnali emessi dal processore nel caso di un sistema con bus dati a 64 bit.

In un sistema con bus dati a 64 bit, la memoria è organizzata per trasferire 8 byte per ciclo di bus. Il processore deve generare 8 segnali BE (BE₀, BE₁, ..., BE₇) per abilitare l'accesso ai singoli byte e la memoria deve essere separata in modo che ogni byte (degli 8) si trovi in dispositivi distinti, in maniera tale da consentire un accesso simultaneo a 64 bit di informazioni contigue.

1.1.51 Spiegare per quale motivo è più efficace utilizzare nel Branch Target Buffer un meccanismo di predizione basato su due bit rispetto a una soluzione con un singolo bit.

All'interno del *Branch Target Buffer* è più conveniente utilizzare un giudizio basato su due bit poiché una predizione potrebbe non essere necessariamente sbagliata al verificarsi della prima eccezione.

Nel caso di cicli annidati, per esempio, il **BRANCH** inerente al loop interno risulta associato ad una predizione falsa ad ogni conclusione del ciclo interno (nonostante gli n casi in cui sia risultata vera). Questo non vuol dire, però, che al ripetersi del ciclo esterno la previsione dello stesso **BRANCH** non torni a risultare vera. Si introducono quindi ulteriori "stati intermedi", oltre a *taken* e *not taken*, per riconsiderare la predizione di un **BRANCH** a più fallimenti della stessa.

1.1.52 Descrivere chiaramente per quale ragione è possibile utilizzare un contatore per lo stadio di fetch del DLX pipelined e quali caratteristiche deve possedere tale contatore.

È possibile utilizzare un contatore a 30 bit come *Program Counter* (PC) nella fase di **FETCH** del DLX pipelined perché le istruzioni sono sempre

allineate a 4 byte, quindi il contatore deve incrementarsi costantemente di +4, il che si traduce in un semplice incremento di 1 sui 30 bit più significativi (dato che i 2 bit meno significativi sono sempre 0). Inoltre, il contatore deve supportare un comando di *LOAD* per aggiornare il valore del PC durante i salti, consentendo così di caricare indirizzi diversi quando necessario.

1.1.53 Spiegare chiaramente cosa sono le eccezioni in un sistema a microprocessore

Per eccezioni in un sistema a microprocessore si intendono eventi interni o esterni che segnalano informazioni alla CPU e quindi ne richiedono l'attenzione. Questi possono essere, per esempio, l'arrivo di un dato in una porta di input (esternamente) oppure una divisione per zero (internamente).

1.1.54 Spiegare cosa è la metastabilità e quali metodologie possono essere adottate per evitarne l'origine.

La metastabilità è una condizione che si verifica quando un segnale viene campionato da un *Flip-Flop* mentre si trova in una fase di transizione o non soddisfa i tempi di setup (*setup time*) o hold (*hold time*) richiesti. In questa situazione, il *Flip-Flop* può entrare in uno stato intermedio in cui la sua uscita non è chiaramente interpretabile come 0 o 1. Questo stato è temporaneo, ma può durare un tempo indefinito.

Una soluzione è data dalla campionatura in due stadi, il primo dei quali campiona il segnale in ingresso ed il secondo il segnale in uscita dal primo

1.1.55 Spiegare le differenze tra la gestione dell'Input/Output mediante polling e interrupt evidenziando vantaggi e svantaggi di ciascuna metodologia.

La gestione dell'Input/Output tramite *polling* prevede che la CPU controlli periodicamente le periferiche per verificarne lo stato. Nel caso degli interrupt, invece, la CPU viene notificata quando un dispositivo necessita di attenzione. Il polling ha il vantaggio di essere semplice ma può rallentare l'esecuzione del codice, poiché la CPU deve continuamente controllare le periferiche. Gli interrupt, d'altra parte, sono più efficienti in termini di utilizzo della CPU, ma richiedono una gestione più complessa di priorità ed identificazione la fonte.

1.1.56 Indicare come è possibile gestire la condizione di stallo di uno stadio di un processore pipelined.

Per gestire la condizione di stallo in una pipeline di un processore DLX, una tecnica semplice consiste nel bloccare l'operazione generatrice e tutte quelle successive, fermandone il clock. Le istruzioni precedenti continuano la loro esecuzione, mentre quelle successive vengono sostituite da operazioni NOP, ed il *Program Counter* non viene incrementato. Tuttavia, esistono altre tecniche più efficienti, come il *Forwarding*, che consente di propagare i dati direttamente tra le fasi della pipeline senza dover aspettare il completamento della scrittura, e la riorganizzazione delle istruzioni, che può essere ottimizzata dal compilatore per ridurre la necessità di stalli.

1.1.57 Con riferimento allo schema del DLX sequenziale mostrato in aula, è possibile effettuare modifiche per ridurre il tempo necessario per eseguire il fetch delle istruzioni?

Sì. Per ridurre le tempistiche della fase di *Fetch* è possibile eliminare il passaggio intermedio tra *Program Counter* e Memoria, ovvero la copia del *Program Counter* all'interno del MAR, effettuando l'accesso direttamente dall'indirizzo segnalato nel registro.

1.1.58 Con riferimento al DLX pipelined: come sono gestiti gli accessi alla memoria di più basso livello nel caso entrambe le memorie IM e DM non contengono le informazioni richieste?

Le caso IM e DM non contengano le informazioni richieste, il *Memory Controller* è incaricato di richiedere tali dati dalla memoria centrale, fornirli alla CPU e inserirli nelle memorie di più basso livello, insieme a quelli subito adiacenti (line-fill).

1.1.59 Con riferimento a istruzioni PC-relative come i branch; eseguendo del codice, potrebbero verificarsi casi nei quali il DLX a non è in grado di portare a termine l'esecuzione di tali istruzioni?

Sì. Un processore non è in grado di portare a termine l'esecuzione di un *BRANCH* quando l'immediato non è un multiplo di 4, il che porterebbe ad un *Program Counter* non allineato

1.1.60 Nel DLX, come avviene l'estensione di un byte signed letto dalla memoria? Spiegare in dettaglio

Nell'architettura del DLX, ad un byte *signed* letto dalla memoria viene esteso il segno concatenandolo al bit più significativo del byte stesso ripetuto 24 volte prima di essere inserito nel registro destinazione.

$$(\text{Byte}[7])^{24} \text{ ## Byte}[7..0]$$

1.1.61 In un sistema reale basato su DLX è sempre necessaria la rete logica che consente di determinare se il sistema è in fase di avvio?

Sì, in un sistema reale è necessaria la rete di avvio. Il DLX, infatti, esegue il codice che si trova all'indirizzo 0h in memoria non solo in fase di interrupt ma anche in fase di avvio. Risulta generalmente necessaria la distinzione dei due casi.

1.2 Esercizi di codice

1.2.1 Sostituire alla label **HERE**, il valore esadecimale corrispondente

```
2000140Ch  HERE: LW R1, R18(0x0100h)
20001404h      ADD R18, R1, R6
20001408h      SUB R12, R1, R3
2000140Ch      BNEQ R12, HERE
```

L'indirizzo di arrivo è 2000140Ch mentre quello di partenza 2000140Ch. La differenza risulta essere di -12 Byte ma bisogna sempre tenere in considerazione l'incremento del PC effettuato all'inizio dell'operazione di **BRANCH**, per cui -16 Byte.

Sapendo che le operazioni di **BRANCH** lavorano con immediati a 16 bit:

```
16      0000 0000 0001 0000
16'     1111 1111 1110 1111
16' + 1 1111 1111 1111 0000
```

Che risulta valere FFF0h

1.2.2 Sostituire alla label **DEST**, il valore esadecimale corrispondente:

```
40001000h  DEST: ADD R1, R0, R7
...
40002000h      JAL DEST
```

L'indirizzo di arrivo è 40001000h mentre quello di partenza 40002000h. La differenza risulta essere di -1000h ma bisogna sempre tenere in considerazione l'incremento del PC effettuato all'inizio dell'operazione di **JUMP AND LINK**, per cui -1004h.

Sapendo che le operazioni di **JUMP AND LINK** lavorano con immediati a 26 bit:

```
1004h      00 0000 0000 0001 0000 0000 0100
1004h'     11 1111 1111 1110 1111 1111 1011
1004h' + 1 11 1111 1111 1110 1111 1111 1100
```

Che risulta valere 3FFEFFCh

1.2.3 Il codice seguente è corretto? Motivare chiaramente la risposta.

```
ADD R7, R0, 0x4321
SUBI R8, R0, 0x0001
SHU R9, (0x1022)R7
AND R10, R9, R8
```

No, il codice non è corretto poiché viene effettuata una STORE HALF-WORD UNSIGNED ad un indirizzo non allineato, ovvero 00005343h (non divisibile per due)

1.2.4 In un DLX senza forwarding unit/cycle-split, indicare quanti stalli risultano con la seguente sequenza di istruzioni mostrando graficamente l'evoluzione temporale della pipeline.

```
ADD R1, R2, R3
LW R2, 0xFF00(R3)
SUB R3, R1, R3
```

	1	2	3	4	5	6	7	8	9
ADD R1, R2, R3	IF	ID	EX	MEM	WB				
LW R2, 0xFF00(R3)		IF	ID	EX	MEM	WB			
SUB R3, R1, R3			IF	*S*	*S*	ID	EX	MEM	WB

1.2.5 Determinare il numero di stalli che si verificano con la seguente sequenza di istruzioni con processore DLX pipelined studiato a lezione in due casi:

- Senza forwarding unit
- Con forwarding unit

```
ADD R1, R4, R5
LHI R5, 0x8000
SB R4, 0x0100(R5)
SB R4, 0x0200(R5)
```

	1	2	3	4	5	6	7	8	9	10
ADD R1, R4, R5	IF	ID	EX	MEM	WB					
LHI R5, 0x8000		IF	ID	EX	MEM	WB				
SB R4, 0x0100(R5)			IF	*S*	*S*	*S*	ID	EX	MEM	WB
SB R4, 0x0200(R5)				*S*	*S*	*S*	IF	ID	EX	MEM

Tabella 1: Senza F.U. - 6 stalli

	1	2	3	4	5	6	7	8
ADD R1, R4, R5	IF	ID	EX	MEM	WB			
LHI R5, 0x8000		IF	ID	EX	MEM	WB		
SB R4, 0x0100(R5)			IF	ID	EX ↓	MEM ↓	WB	
SB R4, 0x0200(R5)				IF	ID	EX ↓	MEM ↓	WB

Tabella 2: Con F.U. - 0 stalli

1.2.6 Indicare quali strategie, hardware e software, è possibile adottare per risolvere eventuali alee generate dalla sequenza di istruzioni seguente:

ADD R7, R8, R9
ADD R1, R2, R3
SUB R4, R5, R6
SUB R10, R7, R20
ADDI R25, R20, 0x0008

	1	2	3	4	5	6	7	8	9
ADD R7, R8, R9	IF	ID	EX	MEM	WB				
ADD R1, R2, R3		IF	ID	EX	MEM	WB			
SUB R4, R5, R6			IF	ID	EX	MEM	WB		
SUB R10, R7, R20				IF	ID	EX	MEM	WB	
ADDI R25, R20, 0x0008					IF	ID	EX	MEM	WB

Il codice presenta un'alea di dato tra l'istruzione 1 e l'istruzione 4, poiché quando l'istruzione 4 si trova nella fase di *DECODE* l'istruzione 1 si trova ancora nella fase di *WB* e quindi non aggiornerà in tempo il valore di R7. Se si volesse adottare una strategia solamente di tipo software, per risolvere lo stallo sarebbe possibile eseguire l'istruzione 5 prima dell'istruzione 4. Queste sono infatti indipendenti e tale spostamento permetterebbe all'istruzione 1 di aggiornare il valore di R7 prima dell'ingresso nella fase di *DECODE* dell'istruzione 4.

Se si cercasse una soluzione di tipo hardware, invece, si potrebbe implementare la tecnica del *Forwarding* o dello *Split Cycle*. La prima eviterebbe qualunque stallo propagando il valore del registro R7 (attualmente in *WB*)

direttamente nella fase di *EXECUTE* dell'istruzione 4 mentre la seconda dividerebbe il 2 fasi separate il clock, in modo tale che quando l'istruzione 1 si trova in *WB* l'istruzione 4 possa già usufruire del valore aggiornato (nella seconda metà del clock)

1.2.7 Si consideri la seguente sequenza di istruzioni assembler DLX. Indicare, mostrando l'evoluzione delle istruzioni nella pipeline, quanti stalli risultano nei seguenti tre casi: senza forwarding unit e senza cycle-split, con forwarding unit e senza cycle-split, senza forwarding unit e con cycle-split

SUB R4, R5, R6
LW R4, 0x0100(R12)
LW R9, 0x0200(R4)
ADD R6, R8, R1
ADDI R15, R4, 0x0001

	1	2	3	4	5	6	7	8	9	10
SUB R4, R5, R6	IF	ID	EX	MEM	WB					
LW R4, 0x100(R12)		IF	ID	EX	MEM	WB				
LW R9, 0x200(R4)			IF	*S*	*S*	*S*	ID	EX	MEM	WB
ADD R6, R8, R1				*S*	*S*	*S*	IF	ID	EX	MEM
ADDI R15, R4, 0x1								IF	ID	EX

Tabella 3: Senza F.U. e senza S.C.

	1	2	3	4	5	6	7	8	9	10
SUB R4, R5, R6	IF	ID	EX	MEM	WB					
LW R4, 0x100(R12)		IF	ID	EX	MEM	WB				
LW R9, 0x200(R4)			IF	*S*	ID	EX↓	MEM	WB		
ADD R6, R8, R1				*S*	IF	ID	EX	MEM	WB	
ADDI R15, R4, 0x1						IF	ID	EX	MEM	WB

Tabella 4: Con F.U. e senza S.C.

	1	2	3	4	5	6	7	8	9	10
SUB R4, R5, R6	IF	ID	EX	MEM	WB					
LW R4, 0x100(R12)		IF	ID	EX	MEM	WB				
LW R9, 0x200(R4)			IF	*S*	*S*	ID	EX	MEM	WB	
ADD R6, R8, R1				*S*	*S*	IF	ID	EX	MEM	WB
ADDI R15, R4, 0x1							IF	ID	EX	

Tabella 5: Senza F.U. e con S.C.

1.2.8 Con riferimento al DLX pipelined non dotato di unità di cycle split; indicare quanti stalli risultano nell'esecuzione delle seguenti istruzioni: senza forwarding unit, con forwarding unit

ADD R1, R0, R0
ADDI R2, R0, 0x0001
SW R2, 0x0010(R1)
SW R2, 0x0014(R1)

	1	2	3	4	5	6	7	8	9	10
ADD R1, R0, R0	IF	ID	EX	MEM	WB					
ADDI R2, R0, 0x0001		IF	ID	EX	MEM	WB				
SW R2, 0x0010(R1)			IF	*S*	*S*	*S*	ID	EX	MEM	WB
SW R2, 0x0014(R1)							IF	ID	EX	MEM

Tabella 6: Senza F.U.

	1	2	3	4	5	6	7	8
ADD R1, R0, R0	IF	ID	EX	MEM	WB			
ADDI R2, R0, 0x0001		IF	ID	EX	MEM	WB		
SW R2, 0x0010(R1)			IF	ID	EX	MEM	WB	
SW R2, 0x0014(R1)				IF	ID	EX	MEM	WB

Tabella 7: Con F.U.

1.2.9 Quale valore assume il registro R2 al termine dell'esecuzione? Quanto vale lo skip?

ADDI R2, R0, 0xA643
ANDI R3, R2, 0x0001
BEQZ R3, skip
ORI R2, R2, 0xFF0
skip : LHI R4, 0x6000

1. $R2 = 0 + (Imm[15])^{16} \# \# 0xA643 = 0xFFFFA643$
2. $R3 = R2[0] = 0x0001$
3. $R3 = 0$ è falso
4. $R2 = R2$ bitwise OR $0xFF0 = 0xFFFFAFF3$

5. Non influenza $R2$

Il valori finali sono: $R2 = 0\text{xFFFFAFF3}$ e $skip = 0\text{x0004}$.